



Universidad Abierta Interamericana

Facultad de Tecnología Informática

Ingeniería en Sistemas Informáticos

TRABAJO FINAL DE INGENIERÍA

Manual de instalación

Puesta en marcha del sistema en un entorno nuevo

Alumno: Pettini, Patricio Ezequiel

Legajo: B00072710-T1

Sede: Sede Norte — Boulogne

Docentes: Romero, Martín Miguel · Weingand, Gastón Matías

Versión 1.0 · 17/09/2026

Índice

Alcance de esta versión	3
Alcance de este manual	4
Requisitos previos	4
Instalación	5
1. Obtener el código	5
2. Ejecutar el script de arranque	5
3. Entrar	5
Usuarios de demostración	6
Modos de ejecución	7
Direcciones	7
Instalación manual	7
Verificación de la instalación	8
Problemas frecuentes	9
Fuentes	10

Alcance de esta versión

Este manual describe la versión **1.0** del sistema, del **17/09/2026**, que es la que se entrega con el Trabajo Final. Documenta lo que el sistema hace hoy, no lo que va a hacer: si una pantalla no aparece acá, es porque todavía no existe.

Al cierre de esta versión el backlog registra **90 de 92 historias terminadas**. Lo que queda abierto está declarado, con su sprint asignado, en el documento de gestión del proyecto.

Sistema: PredictIT — Gestión y mantenimiento predictivo de equipos informáticos

Versión: 1.0

Destinatario: persona con conocimientos de administración de sistemas.

Alcance de este manual

Este manual describe cómo dejar PredictIT funcionando en una máquina, desde cero. Cubre la instalación para desarrollo y evaluación, la que corresponde al alcance del Trabajo Final; el despliegue en un servidor productivo de Azure está declarado fuera de alcance en la sección 10.2.3 del documento.

Al terminar, el sistema queda accesible en el navegador con datos de demostración cargados: dos organizaciones, seis usuarios, 52 equipos y un historial de incidencias y mantenimientos suficiente para que el análisis predictivo tenga con qué trabajar.

Requisitos previos

Componente	Versión	Para qué
Windows	10 o 11	Los scripts de instalación son de PowerShell
.NET SDK	9.0	Compilar y ejecutar la API
Docker Desktop	Cualquiera reciente	Contenedor de SQL Server
Node.js	20 o superior	Compilar y servir el frontend
Git	Cualquiera	Obtener el código

Verificación de que están todos:

```
dotnet --version
docker --version
node --version
```

Si dotnet no aparece pese a estar instalado, es porque no quedó en el PATH de la sesión. El script de arranque lo resuelve solo: busca el ejecutable en C:\Program Files\dotnet antes de darse por vencido.

Docker Desktop tiene que estar corriendo, no solamente instalado. El script verifica que el contenedor de SQL Server responda antes de seguir, pero no puede iniciar Docker Desktop por su cuenta.

Instalación

1. Obtener el código

```
git clone https://github.com/PatricioPettini/trabajo-final-ingenier-a.git TFI
cd TFI
```

2. Ejecutar el script de arranque

```
.\arrancar.ps1
```

El script hace, en este orden:

1. **Verifica las dependencias.** Si falta alguna, se detiene con un mensaje que dice cuál.
2. **Crea `appsettings.Development.json`** a partir de la plantilla `.ejemplo`, con una clave de firma de token generada al azar. Ese archivo no está en el repositorio a propósito: contiene un secreto.
3. **Levanta el contenedor de SQL Server** con `docker compose up -d` si no estaba arriba, y espera a que acepte conexiones. SQL Server tarda entre veinte y cuarenta segundos en estar listo después de arrancar, y conectarse antes falla.
4. **Aplica los esquemas y los datos de demostración** ejecutando los cuatro scripts de db/ en orden.
5. **Compila el backend.**
6. **Instala las dependencias del frontend** si `node_modules` no existe.
7. **Levanta la API y el frontend**, y espera a que la API responda antes de avisar que está lista.

Al terminar imprime las direcciones y los usuarios de demostración.

3. Entrar

Abrir `http://localhost:5173` e iniciar sesión con `admin / Admin.2026`.

Usuario	Contraseña	Perfil	Para qué sirve
admin	Admin.2026	Administrador	Ve todo el sistema
tecnico1	Tecnico.2026	Responsable Técnico	Ve sólo las incidencias que tiene asignadas
solicitante	Usuario.2026	Usuario Solicitante	Módulo del cliente: reporta y sigue sus reportes
partner	Partner.2026	Partner	Administra dos organizaciones y puede alternar entre ellas

Modos de ejecución

```
.\arrancar.ps1           Levanta todo
.\arrancar.ps1 -SoloBase Sólo el contenedor y los scripts de base de datos
.\arrancar.ps1 -IaCaida  El proveedor de IA falla siempre
.\arrancar.ps1 -Detener  Baja la API y el frontend
```

-IaCaida es el modo de demostración del caso de prueba CP-14: fuerza que el proveedor de asignación falle en cada llamada, para poder mostrar en vivo la asignación de respaldo y las tres transiciones del disyuntor. Con seis incidencias registradas seguidas se ve el circuito abrirse al quinto fallo y dejar de llamar al proveedor.

-Detener baja la API y el frontend, pero deja el contenedor de SQL Server corriendo, porque volver a levantarlo cuesta más que el resto junto. Para bajarlo también:

```
docker compose down
```

Direcciones

Servicio	Dirección
Frontend	http://localhost:5173
API	http://localhost:5099
Documentación de la API	http://localhost:5099/swagger
SQL Server	localhost,1433

Instalación manual

Si hace falta ejecutar los pasos de a uno —para diagnosticar, o en un entorno donde el script no aplique— el orden es el siguiente.

Base de datos.

```
docker compose up -d
cd db
.\aplicar.ps1
```

aplicar.ps1 copia los scripts dentro del contenedor y los ejecuta con sqlcmd. Los cuatro son **idempotentes**: se pueden volver a correr sobre una base ya creada sin perder datos. Las columnas que se agregaron después de la primera versión del esquema tienen su migración con guarda, de modo que una base vieja se actualiza sin recrearla.

Configuración de la API. Copiar la plantilla y completar la clave:

```
cd backend\PredictIT.Api
copy appsettings.Development.json.ejemplo appsettings.Development.json
```

La clave de firma tiene que tener **al menos 32 bytes**: HS256 no admite menos. Con una clave más corta la API se niega a arrancar y explica por qué, en lugar de fallar al emitir el primer token con un error de la librería. Para generar una:

```
powershell -c "[Convert]::ToBase64String([byte[]](1..48|{%{Get-Random -Max 256}}))"
```

Backend.

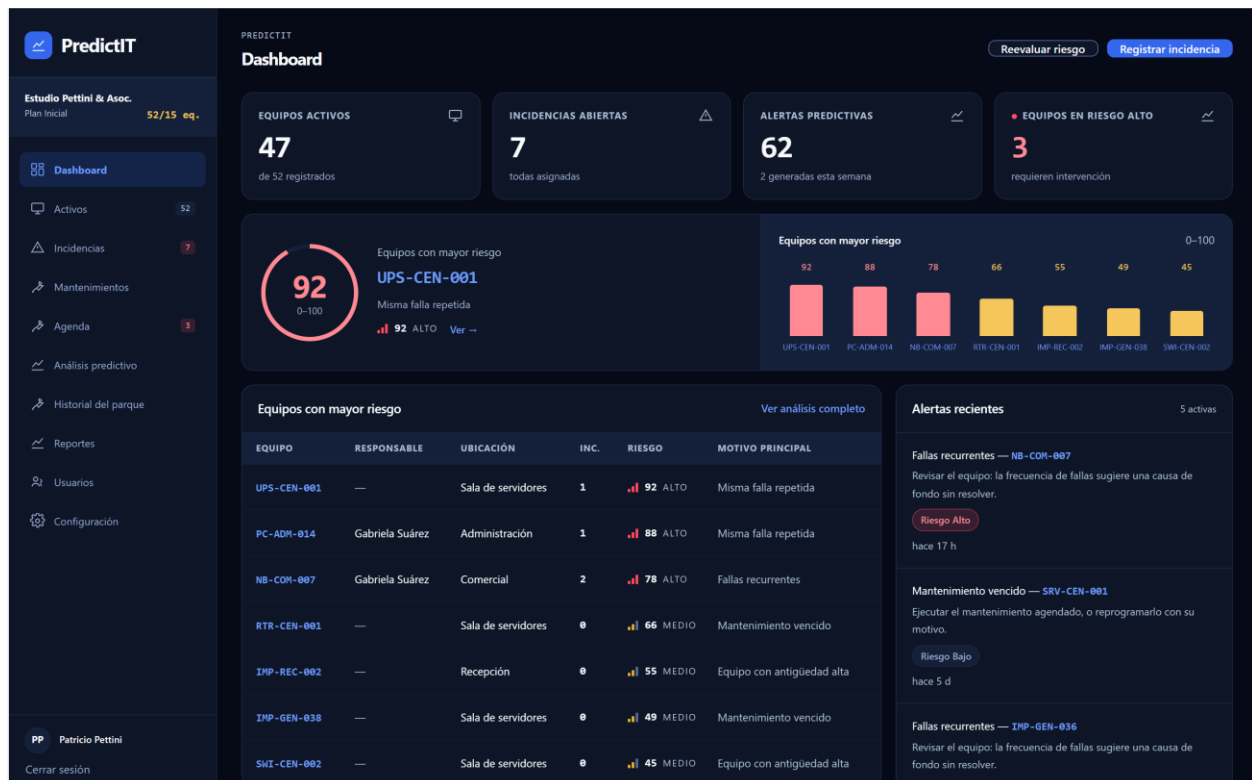
```
cd backend\PredictIT.Api
dotnet run
```

Frontend.

```
cd frontend
npm install
npm run dev
```

Verificación de la instalación

Las pruebas automatizadas son la forma más rápida de confirmar que quedó bien instalado, porque incluyen pruebas de integración que se conectan a la base real.



Las de integración se conectan a la base real como `predictit_app`, así que antes hay que poner su contraseña —la misma que quedó en el `.env`— en el entorno. No tiene valor por defecto a propósito: es la credencial de una base que cada uno crea.

```
$env:PREDICTIT_DB_PASSWORD = "la-que-pusiste-en-el-.env"
dotnet test backend\PredictIT.Tests
cd frontend
npm test
```

La clave de firma del token no hace falta: las pruebas que levantan la API ponen una de usar y tirar si el entorno no trae ninguna. La aplicación sí la exige, y por eso el .env la lleva.

El resultado esperado son 362 pruebas del backend y 91 del frontend, todas en verde. Si fallan las de integración pero pasan las unitarias, el problema está en la base de datos, no en el código.

Problemas frecuentes

«**No se encontró dotnet**» con el SDK instalado. No quedó en el PATH de la sesión. Abrir una terminal nueva, o dejar que el script lo resuelva.

La compilación falla diciendo que un archivo `.dll` está en uso. Quedó una instancia de la API corriendo y tiene el archivo tomado. El script la baja antes de compilar; si se compila a mano:

```
Get-Process -Name PredictIT.Api | Stop-Process -Force
```

El script se queda esperando a SQL Server. Docker Desktop no está corriendo, o el contenedor está arrancando todavía. Ver el estado con:

```
docker ps
docker logs predictit-sql --tail 30
```

«**La bitácora es inmutable**». No es un error de instalación: la tabla de bitácora tiene un disparador que rechaza modificaciones y borrados a propósito. Es el comportamiento correcto.

El frontend carga pero no muestra datos. La API no está respondiendo. Verificar <http://localhost:5099/swagger>. El frontend habla con la API a través del proxy de desarrollo, así que un error de red se ve como pantallas vacías.

Fuentes

Microsoft. (2024). *Documentación de ASP.NET Core y .NET 9*. Microsoft Learn.
<https://learn.microsoft.com/aspnet/core>

Microsoft. (2024). *Documentación de SQL Server 2022*. Microsoft Learn.
<https://learn.microsoft.com/sql/sql-server>